Bachelor's thesis
Fysikaalisten tieteiden kandidaattiohjelma
Theoretical physics

American options in the binomial model

Emil Miska Marius Törmi

June 2, 2026

Supervisor(s): Jaakko Lehtomaa

Examiner(s): Kari Rummukainen

University of Helsinki
Faculty of Science

PL 64 (Gustaf Hällströmin katu 2a)
00014 Helsingin yliopisto

HELSINGIN YLIOPISTO — HELSINGFORS UNIVERSITET — UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Degree programme
Faculty of Science	Fysikaalisten tieteiden kandidaattiohjelma Theoretical physics

Tekijä — Författare — Author

Emil Miska Marius Törmi

Työn nimi — Arbetets titel — Title

American options in the binomial model

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidantal — Number of pages
Bachelor's thesis	June 2, 2026	43

Tiivistelmä — Referat — Abstract

Optioiden tutkiminen on tärkeää, sillä optiot mahdollistavat rahoitusriskien pienentämisen sekä auttavat ymmärtämään rahoitusmarkkinoita. Tämä tutkielma käsittelee, mitä optiot ovat sekä sitä, miten niitä käsitellään matemaattisen mallin avulla. Tässä tutkielmassa käsitellään lisäksi laskennallinen esimerkki tietystä optiosta sekä sitä, miten mallin käyttö muuttuu, kun mallin parametreihin sisältyy estimointivirhettä.

Tutkielman käyttämää matemaattista mallia optioiden tutkimiseen kutsutaan binomimalliksi ja se voidaan jäljittää William Sharpen esitykseen vuodelta 1978. Binomimallia kehitettiin edelleen optioiden hinnoittelua varten Coxin, Rossin ja Rubensteinin toimesta vuonna 1979. Malli pohjautuu äärellisten otosavaruuksien todennäköisyysteoriaan. Todennäköisyysteoreettisen esityksen mallista on esittänyt Shreve teoksessaan vuonna 2004. Tutkielma käyttää tätä teosta päälähteenään. Tutkielma johtaa binomimallin todennäköisyysteoriasta sekä todistaa optioille keskeisimmät tulokset binomimallissa.

Tutkielmassa esitetään optioille sopiva määritelmä sekä todistetaan optioiden kannalta keskeinen teoreema. Tämä teoreema liittyy optioiden hinnoitteluun sekä niiden käyttöön riskienhallinnassa. Teoreemaa käytettään myös laskennalliseen esimerkkiin, jossa on estimointivirheen mahdollisuus. Tutkielman johtopäätös on, että estimointivirhe tekee binomimallista epätarkemman, mutta ei tee siitä käyttökelvotonta. Tutkielmassa käytetyn estimointivirheen malli ei kuitenkaan kuvaa täysin binomimallin kohtaamia käyttöönotto-ongelmia, eikä se kata kaikkia virhetyyppejä.

Avainsanat — Nyckelord — Keywords

Binomial model, American options

Säilytyspaikka — Förvaringsställe — Where deposited

Muita tietoja — Övriga uppgifter — Additional information

Contents

1. Introduction

Options are contracts between the seller and the owner of the option. They can be sold and bought just as stocks, however, they tend to be more complicated and riskier. For financial systems, it is crucial that we know how options work, and how we might mitigate the risk associated with them, since for example, selling options is highly risky. Additionally, our understanding of options might enhance our understanding of financial markets and their efficiency. The binomial asset pricing model is a simple discrete time model that allows us to explore options. It builds a framework for understanding options and their value on a deeper level. The binomial asset pricing model, or the binomial model, can be traced back to a proposal by Sharpe (1978). While Sharpe is thought of as the founder of the model, the model has been expanded by Cox et al. (1979) to better fit the option pricing problem.

In this thesis, we will examine what options are and how can one replicate them, i.e., form a portfolio that always matches the option's value. Specifically, we will investigate how one might replicate path-independent American options in the binomial asset pricing model, or shortly, the binomial model. We aim to give a rigorous mathematical description on how the replication is done, while noting critically the models limitations and exploring some implementation challenges. The binomial model builds itself from a rigorous mathematical foundation in probability theory, especially concepts like finite probability spaces and adapted stochastic processes, which we will likewise explore (Shreve, 2004).

The structure of this thesis is the following. In the beginning of Chapter 2, we will cover the mathematical foundations of the binomial asset pricing model, particularly the concepts we need from probability theory. Later in this chapter, we introduce the binomial asset pricing model and examine it in detail, using different methods and approaches. In Chapter 3, we focus on options as a whole and what does the binomial model imply for them, specifically for the path-independent American option. Furthermore, in this chapter we explore what could simple estimation errors in the coefficients of the binomial asset pricing model mean for the risk mitigation of selling options.

1.1 Use of AI Tools in the Thesis

Claude AI (opus, version 4.6) and Helka's AI search tool were used to find scientific sources. These sources were checked for their scientific legitimacy.

2. The Binomial Model

We first take a look at the mathematical foundations of the binomial model. This chapter introduces many foundations of the binomial model like its probability space.

2.1 Finite Probability Space

The following definitions follow Shreve (2004, Definitions 2.1.1, 2.2.1, 2.3.1, 2.4.1).

Definition 2.1.1 (a finite probability space)**.** *A finite probability space is the tuple* $(\Omega, \mathbb{P})$*.* Ω *is called the sample space. The sample space is a non-empty finite set. The mapping* $\mathbb{P} : \mathcal{P}(\Omega) \to [0, 1]$*, where* $\mathcal{P}(\Omega)$ *is the power set of the sample set, is called the probability measure. We also require that*

$$\sum_{\omega \in \Omega} \mathbb{P}(\{\omega\}) = 1.$$

Additionally, we call subsets of Ω *as events, and we define the probability of the event* $A \in \mathcal{P}(\Omega)$ *as*

$$\mathbb{P}(A) = \sum_{\omega \in A} \mathbb{P}(\{\omega\}).$$

Definition 2.1.2 (the finite probability space of the binomial model)**.** *Let the tuple* $(\Omega_m, \mathbb{P}_m)$*, where* m *is some positive integer, be a finite probability space, where*

$$\Omega_m = \{U, D\}^m$$

and

$$\mathbb{P}_m(\{\omega^{(m)}\}) = p^{N_u(\omega^{(m)})} q^{N_d(\omega^{(m)})},$$

where $\omega^{(m)} \in \Omega_m$*,* $N_u(\omega^{(m)})$ *is the number of* U*s in the sequence* $\omega^{(m)}$ *and* $N_d(\omega^{(m)})$ *is the number of* D*s in the sequence* $\omega^{(m)}$*. Additionally,* $p, q > 0$ *such that* $p + q = 1$*.*

If $m = 0$*, then* $\Omega_0 = \{\omega^{(0)}\}$ *and* $\mathbb{P}_0(\{\omega^{(0)}\}) = 0$*. In particular,* $\omega^{(0)}$ *denotes an empty sequence, i.e., a sequence of length zero.*

Definition 2.1.3 (random variable)**.** *Let the tuple* $(\Omega, \mathbb{P})$ *be a finite probability space, then a random variable* X *defined on this probability space is a function such that* $X : \Omega \to \mathbb{R}$*.*

Definition 2.1.4 (concatenation of sequences)**.** *Suppose we have some two finite sequences* $\omega^{(n)} := (\omega_1, ..., \omega_n) \in \{U, D\}^n$, $\omega^{(m)} := (\omega_{n+1}, ..., \omega_{n+m}) \in \{U, D\}^m$, *where* m, n *are some positive integers, then their concatenation is* $\omega^{(n)} \oplus \omega^{(m)} = (\omega_1, ..., \omega_n, \omega_{n+1}, ..., \omega_{n+m}) \in \{U, D\}^{n+m}$.

In particular, if $\omega^{(0)}$ *is an empty sequence, then* $\omega^{(0)} \oplus \omega^{(n)} = \omega^{(n)} \oplus \omega^{(0)} = \omega^{(n)}$, *for any sequence* $\omega^{(n)} \in \{U, D\}^n$ *and for any positive integers* n.

Definition 2.1.5 (continuation of a sequence)**.** *Let there be a finite set* $\Omega = \{U, D\}^N$ *and a sequence* $\omega^{(k)} = (\omega_1, ..., \omega_k) \in \{U, D\}^k$, *where* $k \leq N$ *and both* k, N *are positive integers. Then a continuation of the sequence* $\omega^{(k)}$ *in the set* Ω *is the sequence* $\omega^{(N-k)} = (\omega_{k+1}, ..., \omega_N) \in \{U, D\}^{N-k}$ *such that the concatenated sequence* $\omega^{(k)} \oplus \omega^{(N-k)} = (\omega_1, ..., \omega_k, \omega_{k+1}, ..., \omega_N) \in \Omega$.

Definition 2.1.6 (adapted stochastic process)**.** *Let the tuple* $(\Omega, \mathbb{P})$ *be a finite probability space defined in Definition 2.1.2 with* $m = N$, *where* N *is some positive integer. We call a sequence of random variables* $X_0, X_1, ..., X_N$ *with each* $X_k : \Omega \to \mathbb{R}$, *for every* $k \in \{0, 1, 2, ..., N\}$, *an adapted stochastic process if, for every* $k \in \{0, 1, 2, ..., N\}$, *every* $\omega^{(k)} \in \{U, D\}^k$ *and any two continuations* $\delta, \delta' \in \{U, D\}^{N-k}$ *of the sequence* $\omega^{(k)}$ *in* Ω, $X_k(\omega^{(k)} \oplus \delta) = X_k(\omega^{(k)} \oplus \delta')$, *where* $\oplus$ *denotes the concatenation of sequences defined in Definition* 2.1.4. *In particular,* X_0 *is a constant.*

2.2 The Market Model and Assets

The binomial asset pricing model can be traced back to William Sharpe's proposal from his book *Investments* from 1978 (Sharpe, 1978). The model was formalized a year later to fit the option pricing problem by Cox, Ross and Rubinstein (Cox et al., 1979). The binomial asset pricing model gives us a straightforward way of replicating options under certain assumptions of the market and of the option in question. In the replication of an option, we build a portfolio whose value matches the value of the option at all times. (Shreve, 2004)

The following definitions follow Shreve (2004, ch. 1).

Definition 2.2.1 (the market model of the binomial model)**.** *The binomial asset pricing model consists of 5 objects, which are treated in discrete time steps indexed by the natural number* $n \in \{0, 1, ..., N\}$, *where we call the positive integer* N *the maturity. These objects are:*

1. *A finite probability space* $(\Omega, \mathbb{P})$ *outlined in Definition* 2.1.2 *with* $m = N$, *where each element of* Ω *represents a full path of the stock price in the time steps from 0 to N. Additionally, the probability measure uses the real probabilities* $p, q > 0$

such that $p + q = 1$. These probabilities are: the probability that the stock price ticks up between two time steps is p and the probability that the stock price ticks down between two time steps is q.

2. *A stock, whose value follows the adapted stochastic process (defined in Definition 2.1.6) $S_0, S_1, ..., S_N$, where $S_n : \Omega \to [0, \infty)$, for every time step $n \in \{0, 1, ..., N\}$.*

3. *An option, whose value follows the adapted stochastic process (defined in Definition 2.1.6) $V_0, V_1, ..., V_N$, where $V_n : \Omega \to [0, \infty)$, for every time step $n \in \{0, 1, ..., N\}$.*

4. *A portfolio, whose value follows the adapted stochastic process $X_0, X_1, ..., X_N$, where $X_n : \Omega \to \mathbb{R}$, for every $n \in \{0, 1, ..., N\}$. Additionally, the amount of the stock in the portfolio follows the adapted stochastic process $\Delta_0, \Delta_1, ..., \Delta_{N-1}$, where $\Delta_n : \Omega \to \mathbb{R}$, for every $n \in \{0, 1, ..., N-1\}$, such that the amount of stock in the portfolio at the time step n is Δ_{n-1} for every $n \in \{1, ..., N\}$. In particular, X_0 is the starting capital and has no stock amount related to it.*

5. *A constant risk-free interest rate $r > 0$ between two time steps n and $n + 1$. For example, if the rate of the money market is r and we invest some amount L in the money market, then at the next time step, we have $(1 + r)L$.*

Furthermore, the binomial asset pricing model makes simplifying assumptions of the market in which options operate in:

1. *Let us have some $\omega^{(n+1)} = \omega^{(n)} \oplus \omega^{(1)}$, where $\omega^{(n)} \in \{U, D\}^n$ and $\omega^{(1)} \in \{U, D\}$. Then, we have for the stock price process $S_0, S_1, ..., S_N$ that*

$$S_{n+1}(\omega^{(n+1)} \oplus \delta) = \begin{cases} uS_n(\omega^{(n+1)} \oplus \delta), & \text{if } \omega^{(n+1)} = \omega^{(n)} \oplus U \\ dS_n(\omega^{(n+1)} \oplus \delta), & \text{if } \omega^{(n+1)} = \omega^{(n)} \oplus D, \end{cases} \tag{2.1}$$

for every continuation δ of $\omega^{(n+1)}$ in Ω and for every $n \in \{0, 1, ..., N-1\}$. Here $u > d > 0$ and we call u the up-factor and d the down-factor. In particular, S_0 is the initial value of the stock, which is a constant.

2. *No-arbitrage, which for the binomial model's stock is equivalent with the inequality*

$$d < r + 1 < u, \tag{2.2}$$

where $u, d > 0$, which are the up- and down-factors of the stock price process; and $r > 0$, which is the risk-free interest rate. We argue, after this definition, why the no-arbitrage assumption should be this in the binomial model.

3. *Regarding investing and loaning, we assume that shares of a stock can be subdivided for sale or purchase, the risk-free interest rate r is the same for loaning and investing, the purchase and selling of a stock happen at the same price, i.e., there is zero bid-ask spread (Shreve, 2004, p. 5). In addition, we are not paying tax or transaction fees (Cox et al., 1979).*

Remark 2.2.1 (evolution of the stock price and its paths)**.** *In Equation* (2.1)*, we make an assumption about the evolution of the stock price. In this equation, we essentially assume that if the last, or most recent, component of the path $\omega^{(n+1)}$ is U, the stock price ticks up by the up-factor u from the price prior to that time step; and if the last, or most recent, component of the path $\omega^{(n+1)}$ is D, the stock price ticks down by the down-factor d from the price prior to that time step. Figure 2.1 illustrates this.*

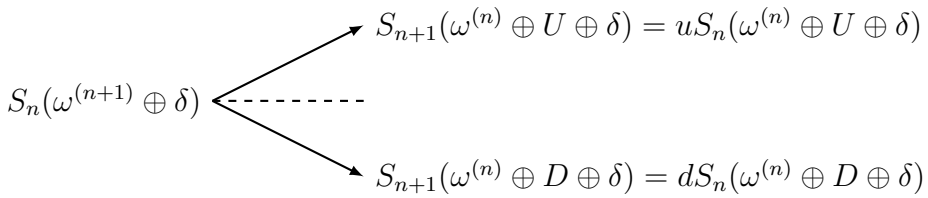

Figure 2.1: Evolution of the stock price S_n, where $\omega^{(n+1)} \in \{U, D\}^{n+1}$, $\omega^{(n)} \in \{U, D\}^n$ such that $\omega^{(n+1)} = \omega^{(n)} \oplus \omega^{(1)}$ for some $\omega^{(1)} \in \{U, D\}$, δ is an arbitrary continuation of $\omega^{(n+1)}$ in Ω and $n \in \{0, 1, ..., N-1\}$.

One could compute the up- and down-factors in the following way

$$u = \frac{S_{n+1}(\omega^{(n)} \oplus U \oplus \delta)}{S_n(\omega^{(n+1)} \oplus \delta)}, \qquad d = \frac{S_{n+1}(\omega^{(n)} \oplus D \oplus \delta)}{S_n(\omega^{(n+1)} \oplus \delta)}. \tag{2.3}$$

Definition 2.2.2 (arbitrage)**.** *Arbitrage is defined as a trading strategy that requires no initial capital, has a zero probability of losing capital and a positive probability of generating capital.*

Remark 2.2.2 (arbitrage in real markets)**.** *Arbitrage does exist in real markets (Shreve, 2004). For example a study of the NYSE (New York Stock Exchange) and the XETRA (eXchange Electronic TRAding) by Rieger et al. (2011) found that there was significant arbitrage opportunities between these two stock exchanges. However, when arbitrage is found and traded upon, the trading removes the arbitrage (Shreve, 2004, p. 2).*

Remark 2.2.3 (an argument for the no-arbitrage Inequality (2.2))**.** *Regarding the no-arbitrage assumption or the 2. assumption of the Definition 2.2.1, if it were that $r + 1 \leq d$, one could loan money from the money market at the interest rate r and invest the loaned money in the stock, which would have the evolution possibilities u and d for which $u > d \geq r + 1$. Thus there would be a zero probability of losing and a*

positive probability of turning to profit. Therefore $u > d \geq r+1$ would create arbitrage. Conversely, if it were that $u \leq r+1$, one could sell the stock short and invest the money in the money market with the interest rate r for which $r+1 \geq u > d$, meaning that there would again be a zero probability of losing and a positive probability of turning to profit, thus $u \leq r+1$ would also introduce arbitrage. Hence, the no-arbitrage assumption should be Equation (2.2)*. (Shreve, 2004, pp. 2-3)*

2.3 One-period Binomial Model

Let us now consider the simplest possible version of the binomial asset pricing model, the one-period version of the model. Suppose we have a binomial market model as defined in Definition 2.2.1 with the maturity $N = 1$, i.e., with one time step. Then the sample space is

$$\Omega = \{U, D\}. \tag{2.4}$$

At the time step $n = 0$ also known as the time zero, the stock is valued at the constant value $S_0(\omega^{(1)})$, for all $\omega^{(1)} \in \Omega$. Additionally, let the option in this model, payoff the amount $V_1(\omega^{(1)})$ at the time step $n = 1$ also known as the time one. For simplicity, we denote $\Delta_0(\omega^{(1)})$ as Δ_0 , $V_0(\omega^{(1)})$ as V_0 and $S_0(\omega^{(1)})$ as S_0, since they are constant and not determined by the path $\omega^{(1)}$. We would now like to find values for the quantities V_0 and Δ_0. We do this by building the replication portfolio, i.e., a portfolio whose value matches that of the option. This portfolio has the value $X_1(\omega^{(1)})$ at the time one. (Shreve, 2004)

The following presentation of the one-period binomial model follows Shreve (2004, pp. 1-8). Suppose that at time zero, the initial capital is X_0 and we buy an amount Δ_0 of the stock at its price S_0. Then we are left with a cash position of the remaining amount $X_0 - \Delta_0 S_0$. Our cash position will evolve with the risk-free interest rate $r > 0$ between time steps. Our stock position will evolve with the stock price. Hence, our wealth at time one will be

$$X_1(\omega^{(1)}) = \Delta_0 S_1(\omega^{(1)}) + (r+1)(X_0 - \Delta_0 S_0), \tag{2.5}$$

where the path the stock price took is $\omega^{(1)} \in \Omega$. However, as mentioned, the price of the stock at time one is not a constant, it is determined by the path $\omega^{(1)}$. As described by the sample set Ω in Equation (2.4), $\omega^{(1)}$ has two possible values, U and D. Thus, our portfolio value $X_1(\omega^{(1)})$ can either be $X_1(U)$ or $X_1(D)$, depending on the path the stock price took. For each of these realized possibilities, we can construct a system of two equations

$$\begin{cases} X_1(U) = \Delta_0 S_1(U) + (r+1)(X_0 - \Delta_0 S_0) \\ X_1(D) = \Delta_0 S_1(D) + (r+1)(X_0 - \Delta_0 S_0). \end{cases}$$

Since we want our portfolio X_n to replicate the option, i.e., we want our portfolio to be a replication portfolio, we set $X_1(U) = V_1(U)$ and $X_1(D) = V_1(D)$. With these constraints, the system of two equations becomes

$$\begin{cases} V_1(U) = \Delta_0 S_1(U) + (r+1)(X_0 - \Delta_0 S_0) \\ V_1(D) = \Delta_0 S_1(D) + (r+1)(X_0 - \Delta_0 S_0). \end{cases} \tag{2.6}$$

If we now multiply the first Equation of (2.6) with a number $\tilde{p}$ and the second equation with $\tilde{q} = 1 - \tilde{p}$ and add these equations together, this results in

$$X_0 + \Delta_0 \left(\frac{1}{r+1}(\tilde{p}S_1(U) + \tilde{q}S_1(D)) - S_0 \right) = \frac{1}{r+1}(\tilde{p}V_1(U) + \tilde{q}V_1(D)). \tag{2.7}$$

We may now define $\tilde{p}$ and $\tilde{q}$ such that

$$S_0 = \frac{1}{r+1}(\tilde{p}S_1(U) + \tilde{q}S_1(D)). \tag{2.8}$$

By the definition of $\tilde{p}$ and $\tilde{q}$ and by the Equation (2.7) we have that

$$X_0 = \frac{1}{r+1}(\tilde{p}V_1(U) + \tilde{q}V_1(D)).$$

Since $S_1(U) = uS_0$ and $S_1(D) = dS_0$ by the 1. assumption of Definition 2.2.1, we may solve Equation (2.8) with respect to $\tilde{p}$ and $\tilde{q}$. This results in

$$\tilde{p} = \frac{1 + r - d}{u - d}, \qquad \tilde{q} = \frac{u - 1 - r}{u - d}.$$

When we subtract the Equations of (2.6) from each other and solve for Δ_0, we get the delta hedging formula

$$\Delta_0 = \frac{V_1(U) - V_1(D)}{(u-d)S_0}.$$

If we were to choose V_0 such that $X_0 > V_0$, then one could short the portfolio and buy the option gaining $X_0 - V_0 > 0$. Since it is guaranteed that $X_1 = V_1$ at every realized outcome in the future, one would gain V_1 and thus could buy out the short position X_1, making a profit of $X_0 - V_0 > 0$ with no initial capital and no risk of losing capital. Thus $X_0 > V_0$ would generate arbitrage. Consider now that we would choose V_0 such that $X_0 < V_0$. Then one could write and sell the option and use that proceed to set up the portfolio, gaining $V_0 - X_0 > 0$. Since $X_1 = V_1$ is again guaranteed at every realized outcome in the future, one could then pay the option's owner with the proceeds from the portfolio, making a profit $V_0 - X_0 > 0$ with no initial capital and no risk of losing capital. Thus, $X_0 < V_0$ would introduce arbitrage. Therefore, to prevent arbitrage, we must to choose V_0 such that $V_0 = X_0$ or equivalently such that

$$V_0 = \frac{1}{r+1}(\tilde{p}V_1(U) + \tilde{q}V_1(D)). \tag{2.9}$$

Furthermore, $\tilde{p}$ and $\tilde{q}$ are both positive and they sum to one. Therefore, we may consider them to be probabilities, which we call, risk-neutral probabilities. Under the actual probabilities $p > 0$ and $q = 1 - p > 0$ of the stock price, we have that

$$S_0(1 + r) < pS_1(U) + qS_1(D),$$

meaning that the average change in the stock price between two time steps, on the right hand side, is strictly more than the change with the risk-free interest rate, on the left hand side. This is due to the fact that if the average change in the stock price would be less than or equal to the rate of change with the risk free interest rate, nobody would want to invest in the stock, since larger or equal returns could be achieved with less risk using the risk-free interest rate (Shreve, 2004, p. 7).

On the contrary, the risk-neutral probabilities satisfy Equation (2.8), where the average change in the stock price is exactly the change with the risk-free interest rate r. Here investors must be neutral regarding risk, meaning they do not want compensation for taking it on nor are they ready to pay to have it. (Shreve, 2004, pp. 7-8)

2.4 General Replication in the Binomial Model

The binomial asset pricing model can be generalized for N number of periods. By doing this, the computational complexity of the option replication problem increases significantly. In this chapter, we examine the multi-period general replication theorem.

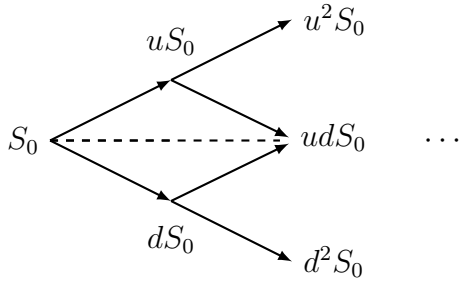

Figure 2.2: A binomial tree of the stock price visualizing the multi-period binomial model.

Definition 2.4.1 (notation of adapted stochastic processes)**.** *To improve readability, we introduce the following notation for all adapted stochastic processes. Every adapted stochastic process $X_0, X_1, ..., X_N$, where $X_n : \Omega \to \mathbb{R}$ for every $n \in \{0, 1, ..., N\}$ (Ω being the sample space of the probability space on which the adapted stochastic processes is defined on), if denoting its input is necessary, we only denote the components of its input which determine its value, otherwise we do not denote its input at all. This means that $X_n(\omega^{(n)} \oplus \delta)$, where $\omega^{(n)} \in \{U, D\}^n$, is denoted by $X_n(\omega^{(n)})$ for all continuations*

δ of $\omega^{(n)}$ in Ω and for any $n \in \{1, ..., N\}$. In particular, since X_0 is a constant, we do not denote any input for it.

The one-period binomial model and the replication done within it, gives us the basic understanding of replication. However, many options, including the American option, require a more general framework for their multi-period replication. Therefore, in this chapter, we introduce the general replication theorem that paves the way for the replication done in Chapter 3. However, before this, we define the risk-neutral probability space and the cash flow process. The following definitions and the theorem and its proof follow theorems and proofs from Shreve (2004, Theorems 2.4.8, 4.2.2).

The risk-neutral probability space is used by the binomial model to formalize the risk-neutral probabilities encountered before. We define it as the following.

Definition 2.4.2 (risk-neutral probability space)**.** *The risk-neutral probability space is a probability space outlined in Definition 2.1.2 with $m = N$, where N is a positive integer called the maturity, and with the risk-neutral probabilities*

$$p = \tilde{p} = \frac{1 + r - d}{u - d}, \quad q = \tilde{q} = \frac{u - 1 - r}{u - d},$$

where u is the up-factor, d is the down-factor and r the risk-free interest rate. For them, $u > r + 1 > d$ and $u, d, r > 0$. We denote this space as the tuple $(\Omega, \tilde{\mathbb{P}})$, where $\tilde{\mathbb{P}}$ stands for the probability measure of Definition 2.1.2 using the risk-neutral probabilities $\tilde{p}, \tilde{q}$.

For the general replication theorem, we need to define a process we call the cash flow.

Definition 2.4.3 (cash flow)**.** *Consider the risk-neutral probability space as defined in Definition 2.4.2 and the binomial market model outlined in Definition 2.2.1. Additionally, consider some functions $v_0, v_1, ..., v_N$, where $v_n : \mathbb{R} \to \mathbb{R}$, for every $n \in \{0, 1, ..., N\}$. We define the cash flow as the adapted stochastic process (as defined in Definition 2.1.6) $C_0, C_1, ..., C_{N-1}$, where $C_n : \Omega \to \mathbb{R}$ for every $n \in \{0, 1, ..., N - 1\}$ such that*

$$C_n = v_n(S_n) - \frac{1}{1 + r}(\tilde{p} v_{n+1}(uS_n) + \tilde{q} v_{n+1}(dS_n)),$$

for every $n \in \{0, 1, ..., N - 1\}$. We call this the cash flow of the functions $v_0, v_1, ..., v_N$.

Theorem 2.4.1 (general replication theorem)**.** *In this theorem and its proof we use the notation convention of Definition 2.4.1. Consider the binomial market model outlined in Definition 2.2.1, the risk-neutral probability space and the maturity N. Let $v_0, v_1, ..., v_N$ be a sequence of functions $v_n : \mathbb{R} \to \mathbb{R}$ such that their cash flow is non-negative, i.e.,*

$$C_n = v_n(S_n) - \frac{1}{1 + r}\left(\tilde{p} v_{n+1}(uS_n) + \tilde{q} v_{n+1}(dS_n)\right) \geq 0 \tag{2.10}$$

for all $n \in \{0, 1, ..., N-1\}$. *Additionally, let us define the delta hedging formula*

$$\Delta_n = \frac{v_{n+1}(uS_n) - v_{n+1}(dS_n)}{(u-d)S_n}, \tag{2.11}$$

for every $n \in \{0, 1, ..., N-1\}$. *If we set* $X_0 = v_0(S_0)$ *and define recursively forward in time the replicating portfolio's values by*

$$X_{n+1} = \Delta_n S_{n+1} + (1+r)(X_n - C_n - \Delta_n S_n), \tag{2.12}$$

for every $n \in \{0, 1, ..., N-1\}$, *then we have that*

$$X_n = v_n(S_n),$$

for all $n \in \{0, 1, ..., N\}$.

Proof of Theorem 2.4.1. We prove Theorem 2.4.1 by induction. The base case $n = 0$, i.e, $X_0 = v_0(S_0)$, holds since we assume it in the Theorem 2.4.1. Our induction assumption is that

$$X_n(\omega^{(n)}) = v_n(S_n(\omega^{(n)})),$$

where $\omega^{(n)} \in \{U, D\}^n$ is arbitrary and fixed. We would then like to prove the induction step, i.e.,

$$\begin{aligned} X_{n+1}(\omega^{(n)} \oplus U) =& v_{n+1}(S_{n+1}(\omega^{(n)} \oplus U)) \\ X_{n+1}(\omega^{(n)} \oplus D) =& v_{n+1}(S_{n+1}(\omega^{(n)} \oplus D)), \end{aligned} \tag{2.13}$$

for this fixed and arbitrary $\omega^{(n)}$. We may start with the first Equation of (2.13) and apply Equation (2.12), which results in

$$\begin{aligned} X_{n+1}(\omega^{(n)} \oplus U) = \Delta_n(\omega^{(n)}) S_{n+1}(\omega^{(n)} \oplus U) \\ + (1+r)(X_n(\omega^{(n)}) - C_n(\omega^{(n)}) - \Delta_n(\omega^{(n)}) S_n(\omega^{(n)})). \end{aligned}$$

Then using the induction assumption $X_n(\omega^{(n)}) = v_n(S_n(\omega^{(n)}))$, we have that

$$\begin{aligned} X_{n+1}(\omega^{(n)} \oplus U) = \Delta_n(\omega^{(n)}) S_{n+1}(\omega^{(n)} \oplus U) \\ + (1+r)(v_n(S_n(\omega^{(n)})) - C_n(\omega^{(n)}) - \Delta_n(\omega^{(n)}) S_n(\omega^{(n)})). \end{aligned}$$

If we now use the 1. assumption of the Definition 2.2.1 and Equation (2.10), we have that

$$\begin{aligned} X_{n+1}(\omega^{(n)} \oplus U) = \Delta_n(\omega^{(n)}) S_n(\omega^{(n)})(u - 1 - r) \\ + \tilde{p} v_{n+1}(uS_n(\omega^{(n)})) + \tilde{q} v_{n+1}(dS_n(\omega^{(n)})). \end{aligned}$$

Then by substituting Equation (2.11), we get

$$X_{n+1}(\omega^{(n)} \oplus U) = \left(\tilde{p} + \frac{u-1-r}{u-d}\right) v_{n+1}(uS_n(\omega^{(n)})) + \left(\tilde{q} - \frac{u-1-r}{u-d}\right) v_{n+1}(dS_n(\omega^{(n)})).$$

Here, from the definitions of $\tilde{p}$ and $\tilde{q}$, we have that

$$\tilde{p} + \frac{u-1-r}{u-d} = 1, \qquad \tilde{q} - \frac{u-1-r}{u-d} = 0.$$

Therefore

$$X_{n+1}(\omega^{(n)} \oplus U) = v_{n+1}(uS_n(\omega^{(n)})),$$

or equivalently, by using the 1. assumption of the Definition 2.2.1

$$X_{n+1}(\omega^{(n)} \oplus U) = v_{n+1}(S_{n+1}(\omega^{(n)} \oplus U)). \tag{2.14}$$

Secondly, we prove the second equation of (2.13). By applying again the Equation (2.12), we get

$$X_{n+1}(\omega^{(n)} \oplus D) = \Delta_n(\omega^{(n)})S_{n+1}(\omega^{(n)} \oplus D) + (1+r)(X_n(\omega^{(n)}) - C_n(\omega^{(n)}) - \Delta_n(\omega^{(n)})S_n(\omega^{(n)})).$$

If we now use the 1. assumption of the Definition 2.2.1 and Equation (2.10), we have that

$$X_{n+1}(\omega^{(n)} \oplus D) = \Delta_n(\omega^{(n)})S_n(\omega^{(n)})(d-1-r) + \tilde{p}v_{n+1}(uS_n(\omega^{(n)})) + \tilde{q}v_{n+1}(dS_n(\omega^{(n)})).$$

Then by substituting Equation (2.11), we get

$$X_{n+1}(\omega^{(n)} \oplus D) = \left(\tilde{p} + \frac{d-1-r}{u-d}\right) v_{n+1}(uS_n(\omega^{(n)})) + \left(\tilde{q} - \frac{d-1-r}{u-d}\right) v_{n+1}(dS_n(\omega^{(n)})).$$

Here, from the definitions of $\tilde{p}, \tilde{q}$ we get that

$$\tilde{p} + \frac{d-1-r}{u-d} = 0, \qquad \tilde{q} - \frac{d-1-r}{u-d} = 1.$$

Therefore

$$X_{n+1}(\omega^{(n)} \oplus D) = v_{n+1}(dS_n(\omega^{(n)})),$$

or equivalently, by using the 1. assumption of the Definition 2.2.1

$$X_{n+1}(\omega^{(n)} \oplus D) = v_{n+1}(S_{n+1}(\omega^{(n)} \oplus D)). \tag{2.15}$$

By Equations (2.14), (2.15) we have that

$$X_{n+1}(\omega^{(n+1)}) = v_{n+1}(S_{n+1}(\omega^{(n+1)})),$$

for an arbitrary $\omega^{(n+1)} \in \{U, D\}^{n+1}$. Therefore the induction step holds. Since the base case $X_0 = v_0(S_0)$ holds, we can conclude, by induction, that

$$X_n(\omega^{(n)}) = v_n(S_n(\omega^{(n)})),$$

for all $n \in \{0, 1, ..., N\}$ and all paths $\omega^{(n)} \in \{U, D\}^n$.

□

3. Options in the binomial model

Options are, by their nature, contracts. These contracts give businesses the opportunity to protect them from types of risk. For example, a farmer might reduce the risk of fertilizer price swings by using options. In addition to risk reduction, options and their pricing could allow us to learn about the markets and their efficiency. (Shreve, 2004)

In the following section, we consider what options are and specifically what are the differences between American and European options. After this introductory part, we explore American options and how they can be replicated in the binomial model, using our theorem about replication established in Chapter 2. Lastly, we introduce a computational example containing a simulation regrading American options and how parameter estimation error might change the actual use of replication for risk reduction.

3.1 European and American Options

The following definitions follow Shreve (2004, chs. 1, 4).

Definition 3.1.1 (a call option)**.** *A call option is a contract between the writer and the owner of the option, which gives its owner the right to buy a specific stock from the writer of the option at a specified strike price K within a specified time frame. In the binomial model the last time step N when the option can be exercised, is called the option's maturity, i.e., the option expires after the maturity.*

Definition 3.1.2 (a put option)**.** *A put option is a contract between the writer and the owner of the option, which gives its owner the right to sell a specific stock to the writer of the option at a specified strike price K within a specified time frame.*

Definition 3.1.3 (an European option)**.** *An European option specifies that the option can be exercised, or used, only at the maturity N.*

Definition 3.1.4 (the intrinsic value of an option)**.** *The intrinsic value of an option is the payoff the owner receives, if they exercise the option immediately. The intrinsic value is represented using a function called the intrinsic value function $g : \mathbb{R} \to \mathbb{R}$, such that $g(S_n)$ is the intrinsic value of the option at the time step n, where S_n is the value of option's stock at the time step n.*

Definition 3.1.5 (an American option)**.** *An American option specifies that the option can be exercised, at any time step n before the maturity and on the maturity N itself, i.e., when the current time step $n \leq N$.*

Example 3.1.1 (American put option)**.** *Consider an American put option with the strike price 100€. Additionally, consider the binomial market model defined in Definition 2.2.1 with $d = 0.95$, $u = 1/d \approx 1.05$ and let the option's maturity $N = 200$. The Figure 3.1 illustrates a single path of the option's stock price under the real probabilities $p = 0.5$, $q = 1 - p = 0.5$ and with the initial stock price of 100€. Since the option is American, the owner can exercise it on time step before the maturity and on the maturity. Suppose then that the owner of the option decides to exercise the option at the time step $n = 30$, when the stock price is approximately 45€. Then the option's owner sells this underlying stock to the option's writer for 100€, since this is the strike price. This is ideal for the option's owner since they sell the stock for 100€ even though its current value is 45€, gaining effectively 55€. Suppose on the contrary, that the option's owner decides exercise the option at the time step $n = 125$, when the stock price is approximately 150€. Then the option's owner again sells the underlying stock to the option's writer for 100€, even though the current value of the stock is 150€. Therefore, this not ideal for the option's owner since they effectively lose 50€.*

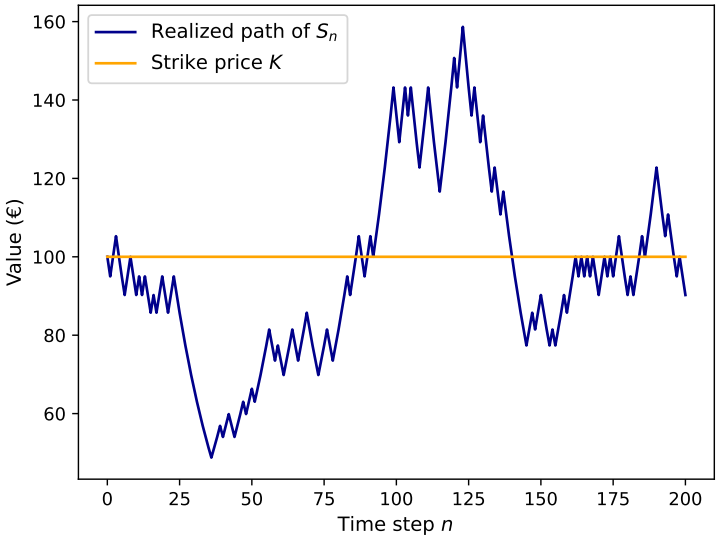

Figure 3.1: A single path of the underlying stock of the American option and the option's strike price. This figure was made using software from Harris et al. (2020) and Hunter (2007).

Remark 3.1.1 (cash flow of American options)**.** *The cash flow defined in Definition 2.4.3 of an American call option (on a non-dividend-paying stock) is never larger than zero (Shreve, 2004, p. 111). This does not hold for an American put option (on a*

non-dividend-paying stock) (Shreve, 2004, p. 113). To clarify, all of the options in this thesis are on non-dividend-paying stocks.

Definition 3.1.6 (a path-independent American option)**.** *A path-independent American option is an American option whose payoff is independent of the path the stock price took, i.e., its payoff is determined completely by the current stock price.*

The intrinsic value is especially interesting in the case of an American option, since there are many time steps the option can be exercised on. Since the intrinsic value should reflect the immediate payoff of the option, we define it for the American option in the following way.

Definition 3.1.7 (intrinsic value of a path-independent American option)**.** *The intrinsic value function $g : \mathbb{R} \to \mathbb{R}$ used to calculate the intrinsic value of an American option depends on whether the option is a call option or a put option. For a path-independent American call option the function has the form*

$$g(s) = \max(s - K, 0),$$

where K is the strike price of the option. Conversely, for a path-independent American put option the function has the form

$$g(s) = \max(K - s, 0),$$

where K is the strike price of the option.

Remark 3.1.2 (an argument for the intrinsic values of path-independent American options)**.** *When we own a path-independent American call option, we have the right, but not the obligation, to buy the stock at the strike price K before and on the maturity. If the stock price then becomes more than the strike price, i.e., at the current time step n, we have that $S_n > K$, then by exercising the option, we could then buy this stock at the strike price K and immediately sell it at the higher price S_n and the option would effectively pay off the difference, i.e., $S_n - K$. But because the stock can also be valued less than the strike price, repeating this process in such a case, we would lose the difference $S_n - K$. It should then be reasonable to assume, that the owner of the option would choose not to exercise the option in such a case, since it would generate a loss for the owner. Therefore, in the time steps where the stock would be valued less than the strike price, the intrinsic value should be zero (due to the fact that we don't have to exercise the option). Thus a path-independent American call option should have the intrinsic value function $g : \mathbb{R} \to \mathbb{R}$*

$$g(s) = \max(s - K, 0).$$

We illustrate the intrinsic value of an American call-option in Figure 3.2.

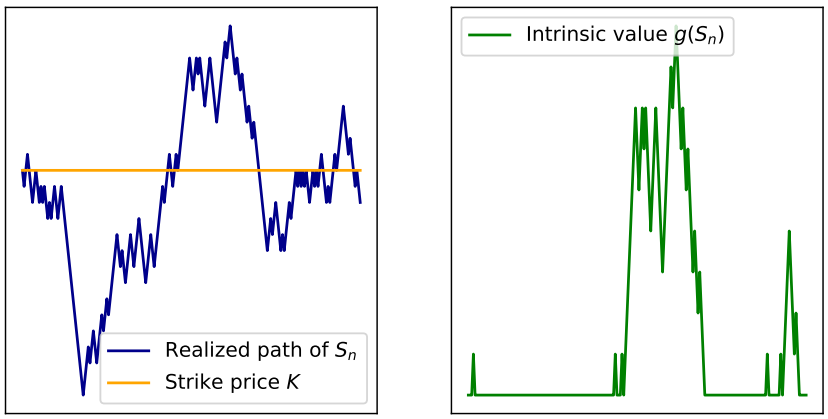

Figure 3.2: A visualization of the intrinsic price of a path-independent American call option. On the left, we have some realized path of the stock price process S_n in dark blue and the options strike price K in orange. On the right, we have the intrinsic value $g(S_n)$ for each realized stock price S_n in green. This figure was done with the software from Harris et al. (2020) and Hunter (2007).

A similar argument applies for put options, but then the option's owner would gain a payoff, when the stock price is below the strike price.

3.2 Replication of Path-independent American Options

We now shift our focus to American options. Specifically, path-independent American options, and their replication, which can be used for risk management. We have so far established a fairly general framework for the replication of options in Theorem 2.4.1. This theorem allows us to replicate an option if we can argue or derive the functions $v_0, v_1, ..., v_N$ for that option. For path-independent American options, these functions are similar to what European options have, but there is a slight change. Let us now define them.

Definition 3.2.1 (the American pricing algorithm)**.** *In the setting of the Theorem 2.4.1, the functions $v_0, v_1, ..., v_N$, where each $v_n : \mathbb{R} \to \mathbb{R}$, for a path-independent American option, are defined such that the value of the path-independent American option V_n is $v_n(S_n)$, for all $n \in \{0, 1, ..., N\}$, with*

$$v_n(s) = \begin{cases} \max(g(s), 0), & \text{for } n = N \\ \max\left(g(s), \frac{1}{r+1}\left(\tilde{p}v_{n+1}(us) + \tilde{q}v_{n+1}(ds)\right)\right), & \text{for } n \in \{0, 1, ..., N-1\}, \end{cases}$$

where $\tilde{p}, \tilde{q}$ are the risk-neutral probabilities defined in Definition 2.4.2, u, d the up- and down-factors of the stock defined in Definition 2.2.1 and $g(s)$ is the intrinsic value function $g : \mathbb{R} \to \mathbb{R}$ of the path-independent American option. (Shreve, 2004, ch. 4)

Remark 3.2.1 (an argument for the American pricing algorithm)**.** *Consider the American pricing algorithm. At the maturity $n = N$, we as the owner of the path-independent American option, can either exercise the option now or then not at all, since it is the last time step, where the option can be exercised. Therefore, at the maturity, the American option's value should either be the payoff we would receive in the immediate exercise, i.e., the intrinsic value, or then zero, for the case that we do not exercise the option. Since any rational owner of the option would choose to maximize their payoff, the option should be valued at the maturity such that its value is the maximum of the intrinsic value and zero.*

Before the maturity, where the current time step $n < N$, the situation becomes more complicated. Again we have two opportunities, either we exercise the option now or then later. By choosing not to exercise the option now, we leave the opportunity for ourselves to exercise the option during the next time step $n + 1$. The value is then either the payoff of the immediate exercise, i.e., the intrinsic value, or then the value of possibly exercising the option at the next time step. Between two time steps the American and the European options are equivalent, given that we do not exercise the American option at time zero. Therefore, when choosing not to exercise the option at the current time step, we should price the American option like a one period European option, where the time zero would be the current time step. This means that the other choice for the price should be the price of an one period European option at time zero or equivalently the price from Equation (2.9) *such that the the time zero corresponds to the current time step n and the time one to the following time step $n+1$. This value at $n + 1$ is also called the continuation value. Rationally, the owner of the option would choose the action that would give them the largest payoff, ergo, the price of the option is the maxima of the intrinsic value and the continuation value, as it is in Definition 3.2.1.*

Using Theorem 2.4.1 and the Definition of the American pricing algorithm 3.2.1, we may replicate a path-independent American option. We close this section with a summarizing corollary that follows from Theorem 2.4.1 and Definition 3.2.1.

Corollary 3.2.1 (replication of path-independent American options)**.** *Consider the binomial market model outlined in Definition 2.2.1 with the risk-neutral measure and the maturity N. Let there be a path-independent American option and let $v_0, v_1, ..., v_N$*

be the functions $v_n : \mathbb{R} \to \mathbb{R}$ *from the Definition 3.2.1, i.e.,*

$$v_n(s) = \begin{cases} \max(g(s), 0), & \text{for } n = N \\ \max\left(g(s), \frac{1}{r+1}\left(\tilde{p}v_{n+1}(us) + \tilde{q}v_{n+1}(ds)\right)\right), & \text{for } n \in \{0, 1, ..., N-1\}. \end{cases}$$

Let us have the cash flow process $C_0, C_1, ..., C_{N-1}$ *for these functions* $v_0, v_1, ..., v_N$ *as defined in Definition 2.4.3, i.e.,*

$$C_n = v_n(S_n) - \frac{1}{1+r}(\tilde{p}v_{n+1}(uS_n) + \tilde{q}v_{n+1}(dS_n)),$$

for every $n \in \{0, 1, ..., N-1\}$*. Define additionally the delta hedging formula as*

$$\Delta_n = \frac{v_{n+1}(uS_n) - v_{n+1}(dS_n)}{(u-d)S_n},$$

for every $n \in \{0, 1, ..., N-1\}$*. If we set* $X_0 = v_0(S_0)$ *and define recursively forward the replicating portfolio's values by*

$$X_{n+1} = \Delta_n S_{n+1} + (1+r)(X_n - C_n - \Delta_n S_n),$$

for every $n \in \{0, 1, ..., N-1\}$*, then we have that*

$$X_n = V_n = v_n(S_n),$$

for all $n \in \{0, 1, ..., N\}$*.*

Proof of Corollary 3.2.1. We prove this Corollary 3.2.1 by referring to Theorem 2.4.1 and Definition 3.2.1. The American pricing algorithm uses the functions $v_0, v_1, ..., v_N$ such that

$$v_n(s) = \begin{cases} \max(g(s), 0), & \text{for } n = N \\ \max\left(g(s), \frac{1}{r+1}\left(\tilde{p}v_{n+1}(us) + \tilde{q}v_{n+1}(ds)\right)\right), & \text{for } n \in \{0, 1, ..., N-1\}. \end{cases}$$

We need to know whether we can apply Theorem 2.4.1 using these functions, since that is what the corollary actually states. Theorem 2.4.1 sets a simple constraint for the functions $v_0, v_1, ..., v_N$, the cash flow defined in Definition 2.4.3 must be non-negative for these functions, i.e,

$$C_n \geq 0,$$

or equivalently

$$v_n(S_n) \geq \frac{1}{1+r}\left(\tilde{p}v_{n+1}(uS_n) + \tilde{q}v_{n+1}(dS_n)\right),$$

for all $n \in \{0, 1, ..., N-1\}$. Here one can see that the maxima structure of the functions $v_0, v_1, ..., v_N$ ensures that this inequality is fulfilled at all time steps $n \in \{0, 1, ..., N-1\}$. Since the inequality holds, we may apply Theorem 2.4.1 for these functions and thus

$X_n = v_n(S_n)$, for all $n \in \{1, ..., N\}$. Additionally, since the Definition 3.2.1 states that $v_n(S_n) = V_n$ for all $n \in \{1, ..., N\}$, we have that $X_n = V_n = v_n(S_n)$, for all $n \in \{0, 1, ..., N\}$.

□

Remark 3.2.2 (an argument for why should $V_n = v_n(S_n)$)**.** *The result of Corollary 3.2.1 states that the replication portfolio's value X_n is equal to $v_n(S_n)$, i.e., $X_n = v_n(S_n)$ for each $n \in \{0, 1, ..., N\}$. Since the replication costs this much to set up, to prevent arbitrage from appearing, we should price the option such that $X_n = V_n$ (as discussed in Section 2.3), i.e., such that $V_n = v_n(S_n)$.*

3.3 Computational Example

Lastly, in this section, we consider a path-independent American option and demonstrate computationally, how would one replicate such an option and why one might do it. For motivation, we consider a bank which writes and sells a path-independent American put option. (The position generated by writing and selling the option is also called a short position in the option.) This creates a risk for the bank, since if the owner of the option exercises the option at a time step, where the intrinsic value is positive, the bank effectively pays a payoff the size of the intrinsic value to the owner of the option. For example, consider that the strike price is 100 € and the current stock price is 90 €. Now the owner chooses to exercise the option, so the bank must buy the stock from the owner for 100 €, even though the current value of the stock was 90 €. Thus, the bank takes a loss of 10 €, which is equivalent to the intrinsic value.

One way the bank can reduce this risk is by replicating the option they just wrote, since this makes the value fluctuations not as extreme. When we set up the replication portfolio for the option, we say that the short position is hedged, or protected. (Shreve, 2004)

The bank's short position in the option motivates the following PnL (profit and loss) Formulae (3.1), (3.2). A PnL formula tells whether the bank's position generated a total profit or a total loss. A negative PnL would indicate a total loss and a positive PnL a total profit. Therefore, if the bank chooses not to hedge their short position in the put option, they sell the option at the time zero at its price V_0, gaining that amount. Subsequently, when the option's owner chooses to exercise the option, the bank loses the intrinsic value at that time step. This results in the PnL formula without the hedge at the execution time step n^*, i.e., the time step where the option's owner decided to exercise the option,

$$\mathrm{PnL}_{n^*} = V_0 - g(S_{n^*}), \tag{3.1}$$

where V_0 is the price of the option at time zero and $g(S_{n^*})$ is the intrinsic value of the option at the time step n^*. Conversely, if the bank chooses to hedge their short position in the option, i.e., the bank sells the option, but immediately after that sets up the replication portfolio with the proceeds from the selling. The PnL formula with the hedged position at the time step n^* then becomes

$$\mathrm{PnL}_{n^*} = X_{n^*} - g(S_{n^*}), \tag{3.2}$$

where X_{n^*} is the value of the replication portfolio at the time step n^*.

Additionally, Cox et al. (1979) suggest that the calculation of u, d and r (the up-factor, the down-factor and the risk-free interest rate) should be done in the following way

$$u = e^{\sigma\sqrt{t/N}}, \quad d = \frac{1}{u} = e^{-\sigma\sqrt{t/N}}, \quad r + 1 = (r_t + 1)^{t/N}, \tag{3.3}$$

where σ is the volatility of the stock price, t a fixed calendar time to the maturity, r_t the risk free interest rate over the whole fixed calendar time t and N the maturity. Here, volatility is the standard deviation of the logarithmic returns, i.e., the differences in the natural logarithm of the stock price between two time steps (Hull, 2011, pp. 498-499). Since volatility must be evaluated from historical data of the stock price, it has an estimation error related to it.

Let us assume that the logarithmic returns are normally distributed and thus sampled from the normal distribution $N(\mu, \sigma^2)$, where μ is the mean logarithmic return and σ is the volatility of the stock price (Hull, 2011, p. 300). Additionally, let the volatility be estimated using k number of observations of logarithmic returns. Then define an auxiliary random variable Z such that

$$Z = \frac{(k-1)s^2}{\sigma^2}, \tag{3.4}$$

where the s^2 is the sample variance of the logarithmic returns (meaning that s is the estimated volatility) and σ is the true volatility. Then it follows that

$$Z \sim \chi^2_{k-1}, \tag{3.5}$$

where χ^2_{k-1} stands for the chi-squared distribution with $k-1$ degrees of freedom (Casella and Berger, 2002, Theorem 5.3.1). This means that Z can be sampled from the chi-squared distribution and by Equation (3.4) we may find the corresponding erroneous volatility estimate s. The value of k again indicates the number of logarithmic return observations we would use in our estimation of the volatility.

Let us now explore what might this error in the volatility do to the bank's PnL. Consider that the true volatility $\sigma = 0.25$, the calendar time is exactly one year, i.e., $t = 1$, the maturity is the 365th time step, i.e., $N = 365$, the strike price of the put

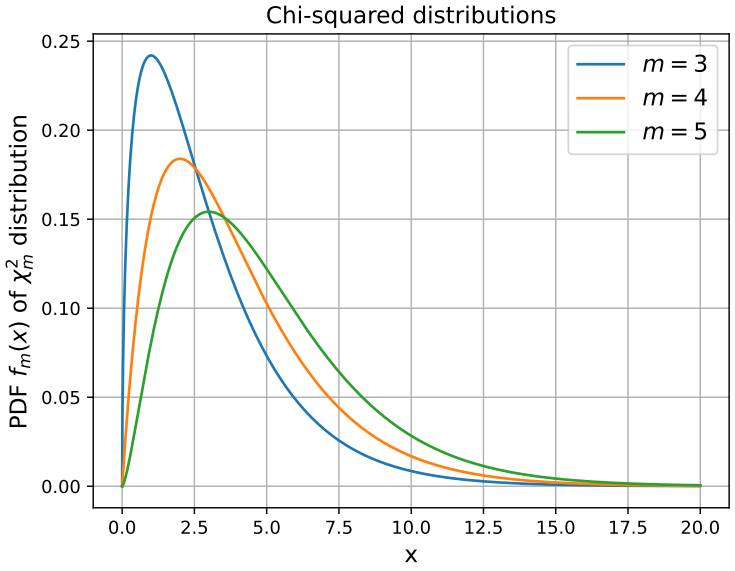

Figure 3.3: Here are some Chi-squared distribution's χ^2_m probability density functions $f_m(x)$, or PDFs, from which the Z variable could be sampled from. This figure was made using software from Virtanen et al. (2020), Harris et al. (2020), Hunter (2007).

option K =100€, the initial stock price S_0 =100€, the risk-free interest rate over the whole calendar time $r_t = 0.05$ and that we use the Equations (3.3) to calculate the true u, true d and r. Suppose then that we have a market as defined in Definition 2.2.1, such that it uses true u, true d and r, i.e., the u, d and r calculated using the true volatility σ, the risk-free interest rate over the whole calendar time r_t and Equations (3.3). Additionally, let the true probabilities of the stock price in this market model be the following. The probability of the stock price moving up between two time steps by the true up-factor u is $p = 0.505$ and the probability of the stock price moving down between two time steps by the true down-factor d is $q = 1 - p = 0.495$.

To find out what kind of effect does the use of the erroneous volatility estimate have on the bank's PnL, we must also assume the behavior of the option's owner. The option's owner only exercises the option, if the cash flow defined in Definition 2.4.3 is strictly positive, i.e., the option's owner exercises the option in the first time step n^* for which $C_{n^*} > 0$. We use this assumption because, if C_{n^*} is positive, it then implies that

$$v_{n^*}(S_{n^*}) > \frac{1}{1+r}\left(\tilde{p}v_{n^*+1}(uS_{n^*}) + \tilde{q}v_{n^*+1}(dS_{n^*})\right),$$

where on the left hand side we have the current value of the option and on the right hand side the value of the option associated with future time steps (also called the continuation value). This inequality implies that the option is currently more valuable than in the future. Thus making the exercise of the option optimal. (Shreve, 2004, p. 114)

In our simulation, we sample a stock price path $S_1, ..., S_N$ using the true probabilities and true up- and down-factors. Then we sample the auxiliary random variable Z, with a specific value of k, i.e., the number of logarithmic return observations used to estimate the volatility, to find the erroneous volatility estimate s with the Equation (3.4). Using the erroneous volatility estimate, we calculate the erroneous up- and down-factors and also the erroneous risk-neutral probabilities. Using these, we calculate the erroneous delta hedge $\Delta_0, \Delta_1, ..., \Delta_{N-1}$ as in Corollary 3.2.1. Using the erroneous delta hedge and the sampled stock price, we find the values for the bank's replication portfolio $X_1, X_2, ..., X_N$, as in Corollary 3.2.1. We then calculate the cash flow process $C_0, C_1, ..., C_{N-1}$, which the option's owner uses to decide when to exercise the option. We do this calculation using the true up- and down-factors. The option's owner thus has better information about the up- and down-factors. Using this cash flow process, we find the execution index n^*, i.e, the first index n^* for which $C_{n^*} > 0$. Lastly, we use the bank's portfolio process $X_1, X_2, ..., X_N$, the sampled stock price path $S_1, S_2, ..., S_N$, the intrinsic value function as defined in Definition 3.1.7, the execution index n^* and the Formulae (3.2), (3.1), to find the PnL of the bank corresponding to this sampled stock price path, with and without the hedge. (If the option is not exercised by the owner, the PnL without the hedge is V_0 and the PnL with the hedge is X_N.) We repeat this process 5000 times using the Python program in Appendix A. We represent the results of the frequencies of the PnLs in the histograms of Figure 3.4 for different values of k in Equations (3.4), (3.5).

From these histograms one can read that, under our assumptions, the PnL of the bank is very unstable without the hedge. On the contrary, when the bank chooses to hedge the short position in the option, even with an erroneous estimate of the volatility, the PnL becomes much more concentrated around zero. If the bank then uses a better estimate of the volatility, i.e., the bank uses more logarithmic return observations to calculate the volatility (meaning that the estimated volatility is calculated by using a larger k in Equations (3.4), (3.5)) then the variation in the PnL further decreases. Even though the PnL distributions without the hedge show that the mean is slightly above zero, they are still much more unstable than those with the hedge.

We conclude that, under our assumptions and with the error in the volatility estimate, the replication still manages to reduce risk, but not with the mathematical precision as Corollary 3.2.1 suggests. It is also important to note, that our assumptions do not hold that well in real markets. For example, logarithmic returns are not always normally distributed. There has been a study by Eden et al. (2017) which investigated the Canadian stock market returns and concluded that stock market returns were better captured by a skewed student's t-distribution rather than our log-normal distribution.

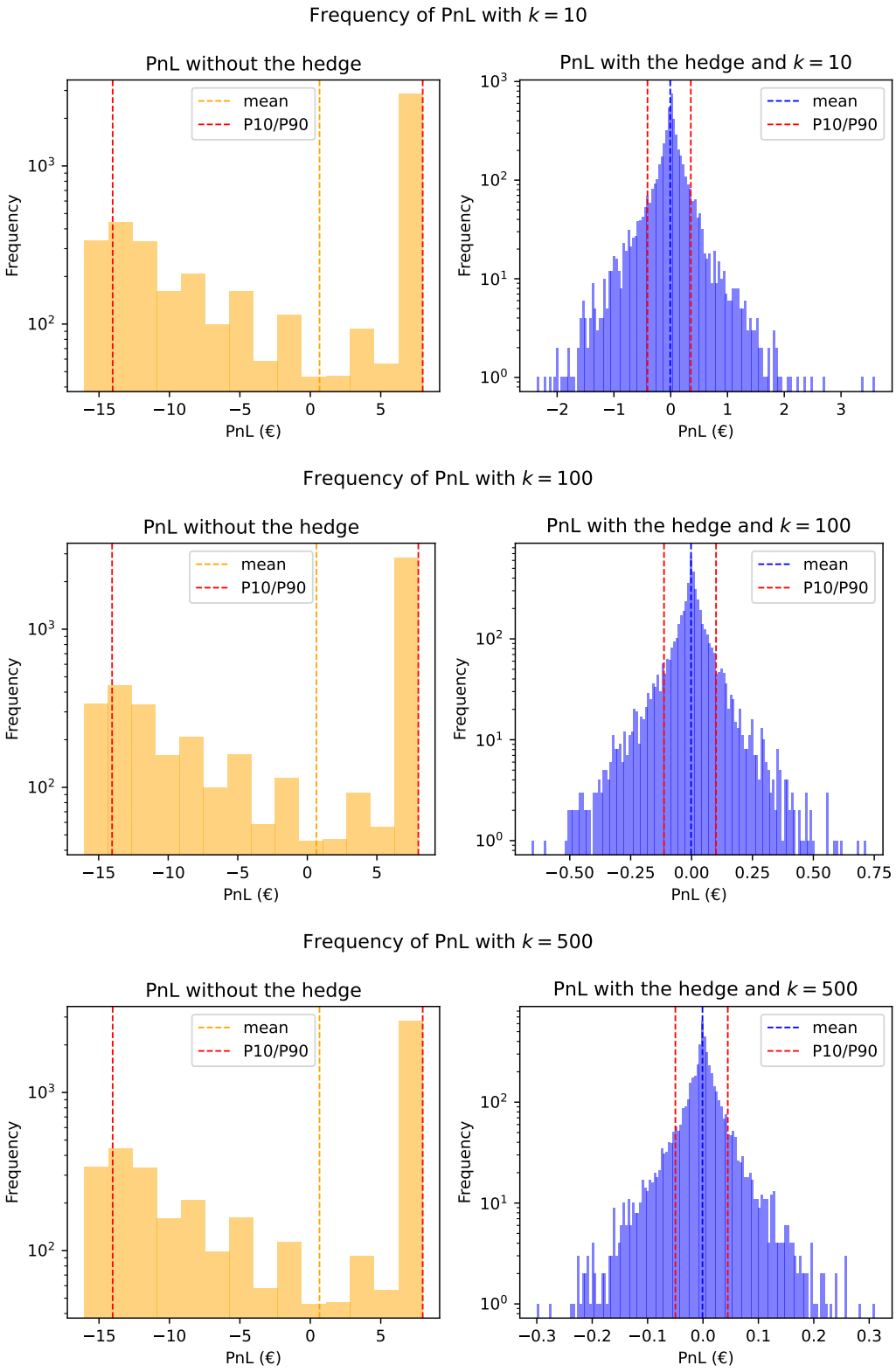

Figure 3.4: Histograms following from the program in the Appendix A. Here we have the PnL of the bank on the x-axis and the frequency of that PnL on the y-axis. The y-axis is in logarithmic scale. Additionally, P10 stands for the 10th percentile and similarly P90 stands for the 90th percentile. Here the number of logarithmic return observations used to estimate the volatility is the value of k as in Equations (3.4), (3.5).

4. Conclusions

This thesis set to cover the binomial model, options and how path-independent American options can be replicated under the binomial model. Additionally, the thesis investigated whether the replication was useful in a short hedging scenario, where the replication was done with erroneous estimates of the up- and down-factors.

The findings were that, under the assumptions of the binomial model, we can replicate path-independent American options as in Corollary 3.2.1 and that even with small estimation errors in the up- and down-factors the replication still manages to be useful in risk management. This means that even if the volatility used to compute the up- and down-factors was only approximate, the replication might still be useful in reducing risk of a short position in a path-independent American option. But as we mentioned, the estimation error does stay relatively low in our demonstration and there might also be other sources of error. More importantly, the assumptions we made about the normality of logarithmic returns does not hold generally for all stocks (Eden et al., 2017). However, our findings do demonstrate that the binomial model can be used for replication of American options and that the replication in turn can be used to reduce the risk of short positions in these options and that the replication is not perfect under parameter estimation error.

To continue where this thesis ends, one could investigate what happens to the PnL under more robust error estimation or what other errors does the binomial model face in the real world and how could those be addressed. For example, how does the assumption of no transaction costs actually impact the models implementation and what kind of error does it introduce. Perhaps transaction costs could be added to the model to make it less error prone. Additionally, the thesis invites the question, that what happens to the replication and its usefulness when some of the assumptions of the binomial model do not hold. For example, what happens when the up- and down-factors are not constant, i.e., when volatility clusters.

Bibliography

Casella, G. and Berger, R. L. (2002). *Statistical inference / George Casella, Roger L. Berger.* Duxbury advanced series. Cengage Learning India Private Limited, Delhi, 2nd ed. edition.

Cox, J. C., Ross, S. A., and Rubinstein, M. (1979). Option pricing: A simplified approach. *Journal of Financial Economics*, 7(3):229–263.

Eden, D., Huffman, P., and Holman, J. (2017). Heavy-tailed distributions and the canadian stock market returns. *Copernican journal of finance & accounting*, 6(2):9–21.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Hull, J. (2011). *Options, futures and other derivatives / John C. Hull.* The Prentice Hall series in finance. Pearson Education, Boston (Mass), 8th ed., global ed. edition.

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Rieger, J., Rüchardt, K., and Vogt, B. (2011). Comparing high frequency data of stocks that are traded simultaneously in the us and germany: Simulated versus empirical data. *Eurasian economic review*, 1(2):126–142.

Sharpe, W. F. (1978). *Investments.* Prentice-Hall, Englewood Cliffs, first edition.

Shreve, S. E. (2004). *Stochastic calculus for finance. 1, The binomial asset pricing model / Steven E. Shreve.* Springer finance. Springer, New York.

The joblib developers (2025). Joblib. Version 1.5.1, Zenodo. https://doi.org/10.5281/zenodo.15496554.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272.

A. Python Implementation of the Computational Example

The following Python code uses software from Virtanen et al. (2020), Harris et al. (2020), Hunter (2007) and The joblib developers (2025).

```python
import random
import numpy as np
import matplotlib.pyplot as plt
import scipy.linalg as scil
from joblib import Parallel, delayed

class AmericanPutSimulator():

    def __init__(
        self,
        initial_stock_price : float,
        total_time : int,
        maturity : int ,
        risk_free_rate_total : float,
        volatility : float,
        real_probability : float,
        number_simulations : int,
        number_observations : int,
        strike_price : float
    ):
        """
        Initializes all variables needed in the simulation.

        Args:
            initial_stock_price: the stock price at time zero. (float)
            total_time: the calendar time to the maturity. (int)
            maturity: the last time step, i.e., the options maturity (int)
```

```
28              risk_free_rate_total: the risk-free interest rate from the
                ↪  total time. (float)
29              volatility: the volatility of the stock price. (float)
30              real_probability: the probability of the up-movement of the
                ↪  stock. (float)
31              number_simulations: the number of times the simulation is
                ↪  repeated. (int)
32              number_observations: the number of observations of logarithmic
                ↪  returns
33              used to estimate the stock price volatilty (int)
34              strike_price: the options strike price (float)
35
36          """
37          self.intrinsic_value = lambda x : (strike_price-x).clip(0)
38          self.initial_stock_price = initial_stock_price
39
40          self.number_simulations = number_simulations
41          self.time_interval = total_time/maturity
42          self.maturity = maturity
43
44          self.number_obervations = number_observations
45          self.volatility = volatility
46
47          self.up_factor = np.exp(volatility*np.sqrt(self.time_interval))
48          self.down_factor = np.exp(-volatility*np.sqrt(self.time_interval))
49          self.risk_free_rate =
            ↪  np.exp(risk_free_rate_total*self.time_interval)
50
51          self.risk_neutral_p = (
52              self.risk_free_rate-self.down_factor
53          )/(self.up_factor-self.down_factor)
54
55          self.risk_neutral_q = 1-self.risk_neutral_p
56          self.real_probability = real_probability
57
58
59      def get_stock_price_tree(self,
60        up_factor : float = None,
61        down_factor : float= None
62        ) -> np.ndarray:
63          """
```

```
64          Builds the stock price tree into a np.ndarray such that it is
            ↪ indexed
65          S[j,n], where n is the time step and j is the number up movements
            ↪ of
66          the stock in that part of the tree.
67
68          Args:
69              up-factor, float, which is the up-factor used to calculate the
70              stock price tree
71              down-factor, float, which is the down-factor used to calculate
                ↪ the
72              stock price tree
73
74          Returns:
75              np.ndarray, which contains the stock price tree.
76
77          """
78          if up_factor is None and down_factor is None:
79              number_of_steps = self.maturity + 1
80
81              # First, a matrix of up-factors
82              up_factor_matrix = np.rot90( np.vander(np.full(
83                  number_of_steps, self.up_factor
84                  ),number_of_steps),k=1, axes=(0, 1))
85
86              # Then a matrix of down-factors
87              column = np.zeros(number_of_steps)
88              column[0] = 1
89              down_factor_matrix = scil.toeplitz(
90                  column, self.down_factor ** np.arange(number_of_steps))
91
92              return (up_factor_matrix * down_factor_matrix *
                ↪ self.initial_stock_price)
93          else:
94              number_of_steps = self.maturity + 1
95
96              # First, a matrix of up-factors
97              up_factor_matrix = np.rot90( np.vander(np.full(
98                  number_of_steps, up_factor
99                  ),number_of_steps),k=1, axes=(0, 1))
100
```

```
101             # Then a matrix of down-factors
102             column = np.zeros(number_of_steps)
103             column[0] = 1
104             down_factor_matrix = scil.toeplitz(
105                 column, down_factor ** np.arange(number_of_steps))
106 
107             return (up_factor_matrix * down_factor_matrix *
                ↪  self.initial_stock_price)
108 
109 
110     def get_option_price_tree(
111         self,
112         up_factor : float = None,
113         down_factor : float= None
114         ) -> np.ndarray:
115         """
116         Builds the option price tree into an Numpu array such that it is
                ↪  indexed
117         V[j,n], where n is the time step and j is the number up movements
                ↪  of the
118         stock in that part of the tree.
119 
120         Args:
121             up-factor, float, which is the up-factor used to calculate the
122             option price tree
123             down-factor, float, which is the down-factor used to calculate
                    ↪  the
124             option price tree
125 
126         Returns:
127             np.ndarray, which contains the option price tree.
128 
129         """
130         if up_factor is None and down_factor is None:
131             stock_price_tree = self.get_stock_price_tree()
132 
133             result = np.zeros((self.maturity+1,self.maturity+1))
134 
135             for inverse_column_index in range(self.maturity+1):
136                 for row_index in range(self.maturity+1):
137 
```

```
                column_index = self.maturity-inverse_column_index

                if row_index > column_index:
                    continue

                if column_index==self.maturity:
                    result[row_index,column_index] =
                    ↪ max(self.intrinsic_value(
                        stock_price_tree[row_index,column_index]
                        ),0)
                else:
                    result[row_index,column_index] = max(
                        self.intrinsic_value(
                            stock_price_tree[row_index,column_index]
                            ),
                        (
                        self.risk_neutral_p*result[
                            row_index+1,column_index+1
                            ]
                        +self.risk_neutral_q*result[
                            row_index,column_index+1
                            ]
                        )/(self.risk_free_rate)
                        )

        return result
    else:
        stock_price_tree =
        ↪ self.get_stock_price_tree(up_factor,down_factor)

        result = np.zeros((self.maturity+1,self.maturity+1))

        risk_neutral_p = (
        self.risk_free_rate-down_factor
        )/(up_factor-down_factor)
        risk_neutral_q = 1 - risk_neutral_p

        for inverse_column_index in range(self.maturity+1):
            for row_index in range(self.maturity+1):

                column_index = self.maturity-inverse_column_index
```

```
177
178                     if row_index > column_index:
179                         continue
180
181                     if column_index==self.maturity:
182                         result[row_index,column_index] =
                            ↪  max(self.intrinsic_value(
183                             stock_price_tree[row_index,column_index]
184                             ),0)
185                     else:
186                         result[row_index,column_index] = max(
187                             self.intrinsic_value(
188                                 stock_price_tree[row_index,column_index]
189                                 ),
190                             (
191                             risk_neutral_p*result[
192                                 row_index+1,column_index+1
193                                 ]
194                             +risk_neutral_q*result[
195                                 row_index,column_index+1
196                                 ]
197                             )/(self.risk_free_rate)
198                             )
199             return result
200
201     def simulate_price(self,seed) -> tuple[np.ndarray, np.ndarray]:
202         """
203         Simulates one path of the stock price.
204
205         Returns:
206             tuple[np.ndarray, np.ndarray], where:
207             1. element is a np.ndarray and contains the single simulated
                ↪  stock
208             price path
209             2. element is a np.ndarray and contains the number of
                ↪  up-movements
210             of the simulated stock price till the current time step.
211
212         """
213         result_price = np.zeros(self.maturity+1)
214         result_number_up_factors = np.zeros(self.maturity+1)
```

```
215
216         for i in range(self.maturity+1):
217             random_choice = np.random.choice(np.array([0, 1]),
218             p=np.array([1- self.real_probability,self.real_probability])
219             )
220             if random_choice==1:
221                 if i == 0:
222                     result_price[i] = self.initial_stock_price
223                     result_number_up_factors[i] = 0
224                 else:
225                     result_price[i] = result_price[i-1]*self.up_factor
226                     result_number_up_factors[i] =
                        ↪  result_number_up_factors[i-1]+1
227             elif random_choice==0:
228                 if i == 0:
229                     result_price[i]=self.initial_stock_price
230                     result_number_up_factors[i] = 0
231                 else:
232                     result_price[i] = result_price[i-1]*self.down_factor
233                     result_number_up_factors[i] =
                        ↪  result_number_up_factors[i-1]
234         return result_price, result_number_up_factors
235
236     def get_delta_hedge(self, seed) -> tuple[
237     np.ndarray,
238     np.ndarray,
239     np.ndarray,
240     ]:
241         """
242         Samples one path of the stock price and then calculates
243         the delta hedge for that path using the error modified up- and
244         down-factors.
245
246         Args:
247             seed: takes in the seed number to initialize np.random.seed
                ↪  (float)
248
249         Returns:
250             tuple[np.ndarray,float,np.ndarray,np.ndarray,np.ndarray] ,
                ↪  where:
251                 1. element is a np.ndarray which contains the delta hedge.
```

```
252                 2. element is a np.ndarray which contains the simulated
                    ↪ stock
253                 price.
254                 3. element is a np.ndarray which contains the number of
255                 up-factors in the stock price path up to the current time
                    ↪ step.
256
257
258         """
259
260         np.random.seed(seed)
261         volatility_with_error = np.sqrt(
262             np.random.chisquare(
263                 self.number_obervations-1
264                 )/(self.number_obervations-1)
265         )*self.volatility
266         up_factor_with_error = np.exp(volatility_with_error
                ↪ *np.sqrt(self.time_interval))
267         down_factor_with_error = 1/up_factor_with_error
268
269         stock_price, number_u_factors = self.simulate_price(seed)
270         option_tree = self.get_option_price_tree(
271             up_factor_with_error,
272             down_factor_with_error
273             )
274
275         result_delta_hedge = np.zeros(self.maturity+1)
276
277         for n in range(self.maturity):
278             result_delta_hedge[n] =
                    ↪ (option_tree[int(number_u_factors[n])+1, n+1]
279             -option_tree[int(number_u_factors[n]), n+1])/(
280                 (
281                     up_factor_with_error-down_factor_with_error
282                     )*stock_price[n]
283             )
284         return result_delta_hedge, stock_price, number_u_factors
285
286     def get_single_pnl_run(self, seed) -> tuple[float, float]:
287         """
288
```

```
289             Simulates one path of the stock price then calculates the
                ↪ delta hedge
290             using the get_delta_hedge method. After this it simulates the
                ↪ PnL of
291             the bank on the simulates path by assuming the behaviour of
                ↪ the
292             option's owner. Particularly, it assumes that the option is
                ↪ exercised
293             if the intrinsic value is larger than the continuation value,
                ↪ i.e,
294             if the cash flow is positive.
295
296             Args:
297                 seed: takes in the seed number to initialize
                    ↪ np.random.seed (float)
298
299             Returns:
300                 tuple[float, float], where:
301                 1. element is the PnL of the bank, without the hedge.
302                 2. element is the PnL of the bank, with the hedge.
303
304         """
305         option_tree = self.get_option_price_tree()
306         initial_option_price = option_tree[0,0]
307         delta, stock_price, number_u_factors = (
308             self.get_delta_hedge(seed)
309             )
310         replication_value = np.zeros(self.maturity+1)
311         replication_value[0] = initial_option_price
312         execution_index = -1
313
314         for time_step in range(1,self.maturity+1):
315
316             if time_step != self.maturity:
317                 cash_flow = option_tree[int(number_u_factors[time_step]),
                    ↪ time_step]-(
318                     self.risk_neutral_p*option_tree[int(
319                         number_u_factors[time_step]
320                         )+1, time_step+1]
321                     +self.risk_neutral_q*option_tree[
322                         int(number_u_factors[time_step]), time_step+1
```

```
                        ]
                )/(self.risk_free_rate)
            else:
                cash_flow = self.intrinsic_value(stock_price[time_step])

            if cash_flow > 0:
                execution_index = time_step
                portfolio_at_execution = (
                    delta[time_step-1]*stock_price[time_step]
                )+(
                    self.risk_free_rate
                    )*(
                    replication_value[time_step-1]
                    -delta[time_step-1]*stock_price[time_step-1]
                    )
                break

            replication_value[time_step] = (
                delta[time_step-1]*stock_price[time_step]
            )+(
                self.risk_free_rate
            )*(
                replication_value[time_step-1]
                -delta[time_step-1]*stock_price[time_step-1]
                )

        if execution_index == -1:
            no_hedge_pnl = initial_option_price
            hedge_pnl = replication_value[-1]
        else:
            no_hedge_pnl = initial_option_price - self.intrinsic_value(
                stock_price[execution_index]
                )
            hedge_pnl = portfolio_at_execution - self.intrinsic_value(
                stock_price[execution_index]
                )

        return no_hedge_pnl, hedge_pnl

    def get_pnl_histograms(self):
        """
```

```
364         Runs the get_single_pnl_run method number_simulation times using
365         Joblib's parallel computing functionality. Then plots histograms
            ↪ of
366         these using Matplotlib.
367
368         """
369         result_no_hedge = []
370         result_hedge = []
371         np.random.seed(0)
372         seeds = np.random.choice(range(1,self.number_simulations*10),
373          size=self.number_simulations, replace=False)
374
375
376         def task(seed):
377             run_no_h, run_h = self.get_single_pnl_run(seed)
378             return (run_no_h, run_h)
379
380         results_list = Parallel(n_jobs=-1)(
381             delayed(task)(seeds[i]) for i in
                ↪ range(self.number_simulations)
382             )
383
384         result_no_hedge, result_hedge = zip(*results_list)
385
386         result_no_hedge = np.array(result_no_hedge)
387         result_hedge = np.array(result_hedge)
388
389         fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 4))
390         fig.suptitle(
391             f'Frequency of PnL with $k=${self.number_obervations}',
392             fontsize=12
393         )
394
395         ax1.hist(result_no_hedge, bins='auto',
396         color='orange', alpha=0.5, log=True
397         )
398
399         ax1.axvline(result_no_hedge.mean(), color='orange',
400         linestyle='dashed', linewidth=1, label='mean'
401         )
402
```

```
        ax1.axvline(np.percentile(result_no_hedge,10), color='red',
        linestyle='dashed', linewidth=1, label='P10/P90'
        )

        ax1.axvline(np.percentile(result_no_hedge,90), color='red',
        linestyle='dashed', linewidth=1
        )

        ax2.hist(result_hedge, bins='auto',
        color='blue', alpha=0.5, log=True
        )

        ax2.axvline(result_hedge.mean(), color='blue',
        linestyle='dashed', linewidth=1, label='mean'
        )

        ax2.axvline(np.percentile(result_hedge,10), color='red',
        linestyle='dashed', linewidth=1, label='P10/P90'
        )

        ax2.axvline(np.percentile(result_hedge,90), color='red',
        linestyle='dashed', linewidth=1
        )


        ax1.set(xlabel='PnL (€)', ylabel='Frequency',
        title='PnL without the hedge'
        )
        ax1.legend( )

        ax2.set(xlabel='PnL (€)', ylabel='Frequency',
        title=f'PnL with the hedge and $k=${self.number_obervations}'
        )
        ax2.legend( )
        plt.tight_layout()
        fig.tight_layout()



simulation = AmericanPutSimulator(
    initial_stock_price=100,
```

```
444     total_time=1,
445     maturity=365,
446     risk_free_rate_total=0.05,
447     volatility=0.25,
448     real_probability=0.505,
449     number_simulations=5000 ,
450     number_observations=10,
451     strike_price=100
452     )
453 
454 simulation.get_pnl_histograms()
455 
456 plt.show()
```